



**TECNOLÓGICO
NACIONAL DE MÉXICO**

INSTITUTO TECNOLÓGICO SUPERIOR DE JEREZ

Carrera: Ingeniería en Sistemas Computacionales

Nombre de la Materia: Tópicos Selectos Web

**Nombre de la Actividad: Reporte de Practica
(SESSION en el servidor)**

Nombre del Alumno(a): Nayeli Morales García

Número de Control: S19070105

Semestre: 14

Nombre del Docente: Salvador Acevedo Sandoval

Fecha de Entrega: viernes 01 de mayo del 2026

Introducción

En esta práctica se desarrolló una aplicación web utilizando Node.js, Express y express-session con el objetivo de comprender el funcionamiento de las sesiones del lado del servidor. Las sesiones permiten almacenar información temporal de los usuarios que interactúan con una aplicación, facilitando el control de acceso, la autenticación y la personalización de la experiencia dentro del sistema.

Durante el desarrollo se implementó un sistema básico de inicio y cierre de sesión, donde el servidor valida las credenciales del usuario y guarda la información de la sesión de manera segura. A diferencia de las sesiones del lado del cliente, en este caso los datos sensibles permanecen almacenados en el servidor, aumentando la seguridad y el control sobre la información.

Además, se utilizó una interfaz sencilla en HTML junto con JavaScript y fetch para enviar solicitudes al servidor y mostrar las respuestas correspondientes al usuario. Esta práctica permitió reforzar conocimientos sobre el manejo de rutas, peticiones HTTP, middleware y administración de sesiones en aplicaciones web modernas.

Desarrollo

Primero creamos la carpeta para la práctica, luego abrimos el editor de texto que utilizamos y abrimos una terminal, entramos a la carpeta que creamos para el proyecto. Luego inicializamos nuestro proyecto con el comando `npm init -y` como se puede ver en la imagen siguiente.

```
PS C:\Users\Wayeli\Desktop\Topicos Programacion WEB\Talleres> cd SesionesEnElServidor
PS C:\Users\Wayeli\Desktop\Topicos Programacion WEB\Talleres\SesionesEnElServidor> npm init -y
Wrote to C:\Users\Wayeli\Desktop\Topicos Programacion WEB\Talleres\SesionesEnElServidor\package.json:

{
  "name": "sesionesenelservidor",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Después de esto instalamos el framework con el que vamos a trabajar en esta practica que en este caso es express.

```
PS C:\Users\Nayeli\Desktop\Topicos Programacion WEB\Talleres\SesionesEnElServidor> npm install express express-session

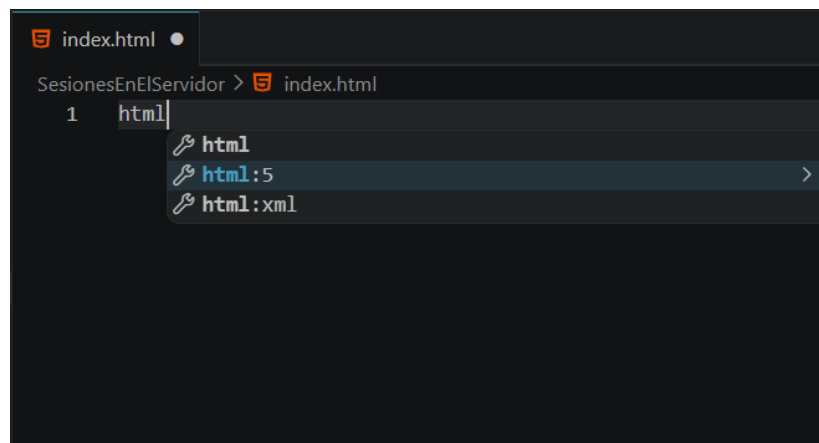
added 73 packages, and audited 74 packages in 20s

24 packages are looking for funding
  run `npm fund` for details

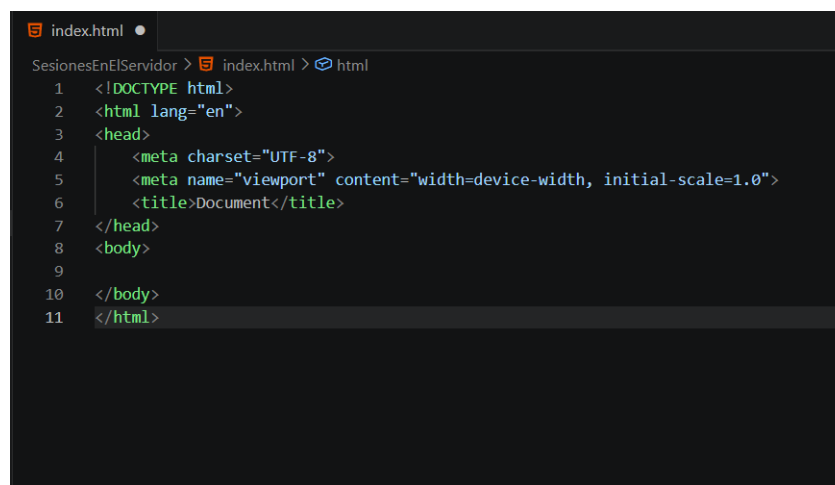
found 0 vulnerabilities
```

Ya que estuvo la instalación y se crearon los json en la carpeta de la práctica. Creamos una carpeta llamada public. Dentro de esta carpeta ahí que crear un archivo que se llama index.html.

Dentro de este agregamos código para estructura de HTML 5, al teclear HTML sale la opción y le damos enter y se agrega el código siguiente.



The screenshot shows a VS Code editor window with a file named 'index.html'. The cursor is at the beginning of the first line, and the text 'html' has been typed. A code completion menu is visible, showing three options: 'html', 'html:5', and 'html:xml'. The 'html:5' option is currently selected.



The screenshot shows the same VS Code editor window with the 'index.html' file. The code completion menu is no longer visible, and the full HTML5 boilerplate code has been entered into the file. The code is as follows:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Document</title>
7 </head>
8 <body>
9
10 </body>
11 </html>
```

Agregamos un título que en este caso es de la practica Session en el lado del servidor. Y agregamos dentro del body el siguiente código que está en la imagen.

```
<body>

  <h2>Sesiones en el servidor</h2>

  <input type="text" id="usuario" placeholder="Usuario">
  <input type="password" id="password" placeholder="Contraseña">

  <button onclick="login()">Iniciar Sesión</button>
  <button onclick="logout()">Cerrar Sesión</button>

  <h2 id="resultado"></h2>
```

Este código su función es de un pequeño formulario con su respectivo título, las cajas de texto donde va la información de el Usuario y el Password. Y también sus botones con su función al momento de hacer click y también va a aparecer el apartado de resultados.

Ahora mismo dentro del body creamos un script con el siguiente código que se muestra en la imagen siguiente.

```
<script>
  async function login() {
    const res = await fetch('/login', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      credentials: 'include',
      body: JSON.stringify({
        usuario: document.getElementById('usuario').value,
        password: document.getElementById('password').value
      })
    });

    const data = await res.json();
    document.getElementById('resultado').innerText = data.mensaje;
  }

  async function logout() {
    const res = await fetch('/logout', {
      method: 'POST',
      credentials: 'include'
    });

    const data = await res.json();
    document.getElementById('resultado').innerText = data.mensaje;
  }
</script>
```

Dentro de la etiqueta `<script>` se encuentran las funciones `login()` y `logout()`, las cuales se ejecutan al presionar los botones correspondientes de iniciar sesión y cerrar sesión.

La función `login()` se encarga de enviar una solicitud a la ruta `/login` utilizando el método `POST`. Para ello, usa `fetch()` y envía en formato `JSON` los datos capturados en los campos de usuario y contraseña (`usuario` y `password`). Además, con `credentials: 'include'` se permite el manejo de cookies o sesiones del usuario.

Cuando el servidor recibe esta petición, valida que las credenciales sean correctas. Si el usuario y la contraseña coinciden, se permite el acceso al sistema. Después, la respuesta del servidor se convierte a formato `JSON` mediante `res.json()`, y el mensaje recibido se muestra en el elemento con `id resultado`, informando si el inicio de sesión fue exitoso o no.

Por otro lado, la función `logout()` se encarga del cierre de sesión. Al hacer clic en el botón correspondiente, se envía una petición `POST` a la ruta `/logout`, también usando `fetch()` y manteniendo las credenciales de sesión con `credentials: 'include'`.

El servidor procesa esta solicitud y finaliza la sesión activa del usuario. Finalmente, la respuesta también se convierte a `JSON` y se muestra en pantalla mediante `data.mensaje`, indicando si el cierre de sesión se realizó correctamente.

Ahora en la carpeta raíz creamos el archivo `server.js` con el código siguiente.

```
server.js X
SesionesEnElServidor > server.js > ...
1  const express = require('express');
2  const session = require('express-session');
3  const app = express();
4
5  app.use(express.static('public'));
6  app.use(express.json());
7
8  app.use(session({
9    secret: 'clave',
10    resave: false,
11    saveUninitialized: false,
12    cookie: {
13      maxAge: 1000 * 60 * 30 // 30 minutos
14    }
15  }));
16
17 // Ruta de login
18 app.post('/login', (req, res) => {
19   const { usuario, password } = req.body;
20
21   if (usuario === 'Nayeli' && password === '1234') {
22     req.session.usuario = usuario;
23
24     res.json({
25       mensaje: 'Sesión iniciada, Bienvenido ' + usuario
26     });
27   } else {
28     res.json({
29       mensaje: 'Usuario o contraseña incorrectos'
30     });
31   }
32 });
33
34 // Ruta de logout
35 app.post('/logout', (req, res) => {
36   req.session.destroy(() => {
37     res.clearCookie('connect.sid', {
38       path: '/'
39     });
40
41     res.json({
42       mensaje: 'Sesión cerrada'
43     });
44   });
45 });
46
47 const PORT = 3000;
48
49 app.listen(PORT, () => {
50   console.log(`Servidor corriendo en http://localhost:${PORT}`);
51 });
```

Este código en server.js tiene la función de crear un servidor con Node.js usando Express y express-session para manejar el inicio y cierre de sesión de un usuario. Primero se importan las librerías necesarias y se configura la carpeta public, donde se encuentran los archivos del lado del cliente, además de permitir que el servidor reciba datos en formato JSON. Después se configura la sesión con una clave secreta, se establece que no se guarden sesiones vacías y se define que la duración de la sesión será de 30 minutos.

La ruta /login recibe una petición POST con el usuario y la contraseña enviados desde el formulario. El servidor verifica si las credenciales coinciden con los datos establecidos (Nayeli y 1234); si son correctas, guarda el usuario en la sesión y

responde con un mensaje de bienvenida indicando que la sesión fue iniciada correctamente. Si los datos son incorrectos, devuelve un mensaje de error.

La ruta /logout también recibe una petición POST y se encarga de cerrar la sesión activa del usuario. Para ello destruye la sesión guardada, elimina la cookie de autenticación y responde con el mensaje de que la sesión fue cerrada. Finalmente, el servidor se ejecuta en el puerto 3000 y queda disponible en <http://localhost:3000>.

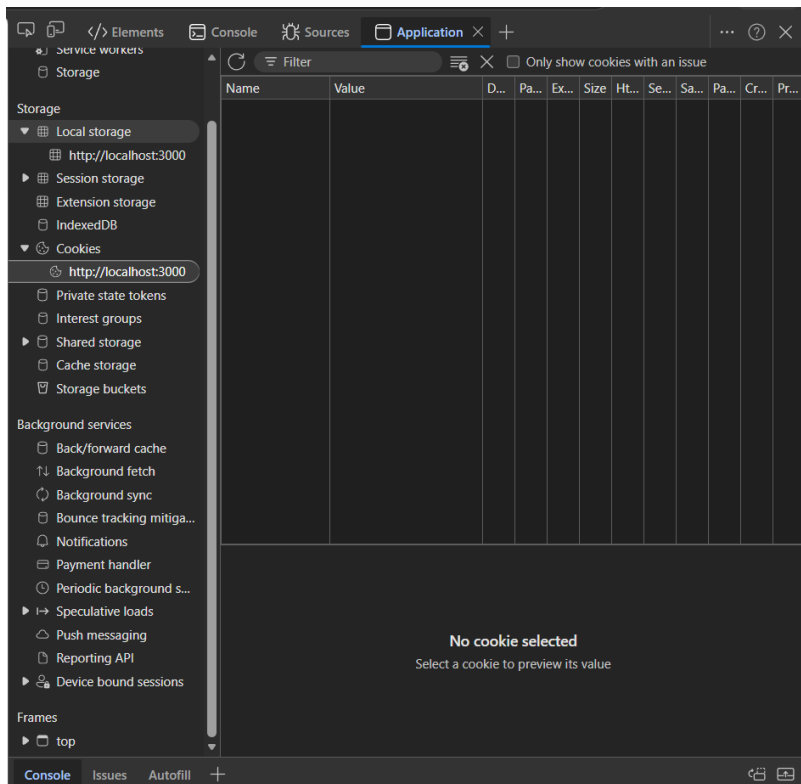
Luego de ya tener el código listo, abrimos la terminal verificamos que estemos dentro de la carpeta del proyecto, tecleamos `node server.js` para correr el servidor. Y se muestra esto de la siguiente imagen.

```
PS C:\Users\Nayeli\Desktop\Topicos Programacion WEB\Talleres\SesionesEnElServidor> node server.js
Servidor corriendo en http://localhost:3000
```

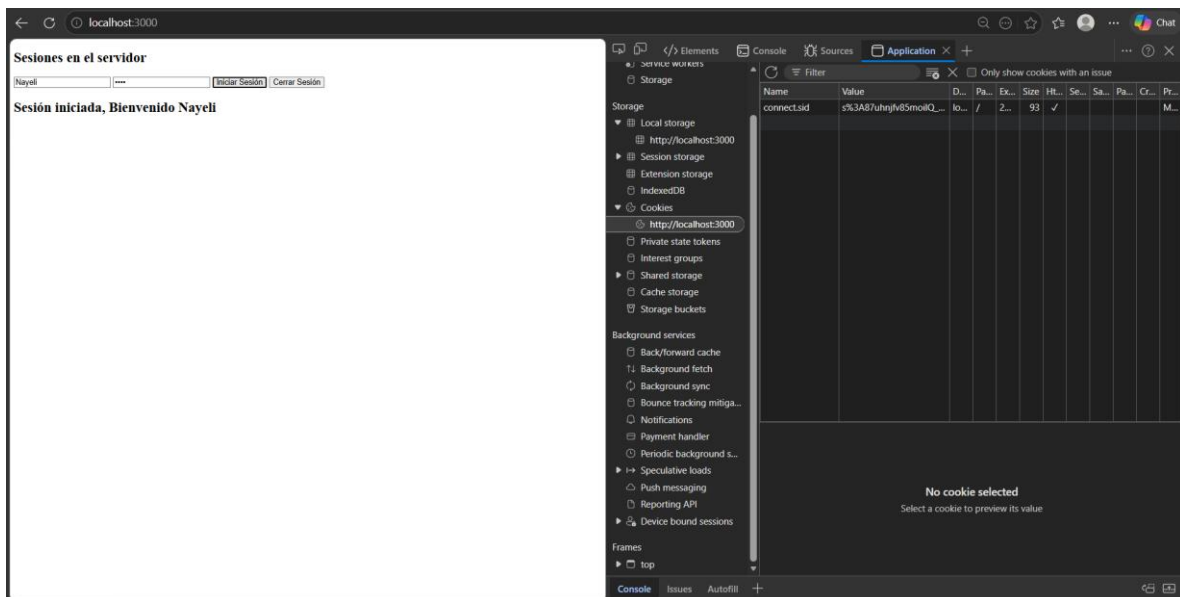
Al parecer esta funcionando correctamente, damos ctrl y click derecho del mouse en el enlace y entramos a nuestra pequeña página.



Aquí en cualquier espacio de la página damos click derecho en inspeccionar, luego en la ventana que aparece buscamos donde dice application y le damos click. Ahora en el menú izquierdo que aparece buscamos cookie y damos click nos muestra algo así.

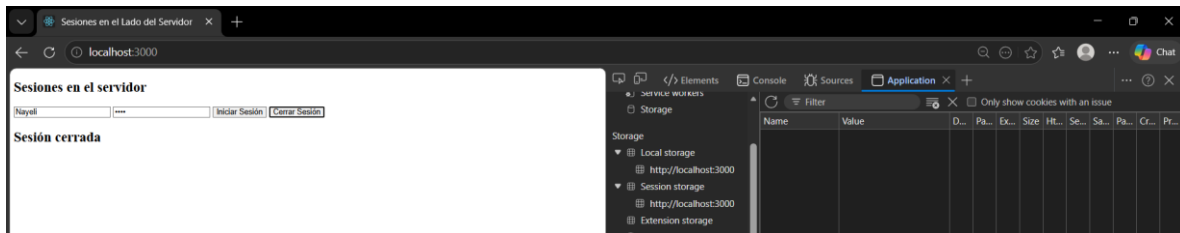


Ahora llenamos en nuestro formulario las cajas de texto con el usuario y contraseña y damos click en iniciar sesión y se aparece esto.



Al iniciar sesión correctamente, aparece la cookie connect.sid porque el servidor usa express-session para manejar la sesión del usuario. Cuando se validan bien el usuario y la contraseña, se crea una sesión y se genera automáticamente esa cookie, que guarda un identificador único de sesión. Esta cookie no almacena el

usuario ni la contraseña, sino una clave que permite al servidor reconocer al usuario en futuras peticiones y mantenerlo autenticado. Al cerrar sesión, esa cookie se elimina junto con la sesión guardada en el servidor.



Y al darle cerrar sesión pues se quita la sesión que estaba con sus datos.

Conclusion

En esta práctica se logró comprender el funcionamiento del manejo de sesiones en el servidor utilizando Node.js, Express y express-session. Se pudo observar cómo el servidor valida las credenciales del usuario mediante una petición POST, permitiendo el acceso solo cuando los datos son correctos. Además, se entendió la importancia de las cookies, especialmente connect.sid, ya que esta permite identificar al usuario y mantener su sesión activa sin necesidad de volver a iniciar sesión en cada petición.

También se comprobó el funcionamiento del cierre de sesión mediante la destrucción de la sesión y la eliminación de la cookie almacenada en el navegador. Esto permitió entender mejor cómo se administra la autenticación de usuarios del lado del servidor y cómo se protege el acceso a la información. En conclusión, esta práctica ayudó a reforzar el uso de sesiones como una herramienta fundamental para la seguridad y el control de acceso en aplicaciones web.